

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

a) Write a C code to create a process using fork() system call.

b) Read the user input as shell command.

c) Use a system call exec() to execute the shell command.

e) Display the PIDs of parent and child processes.

```
b.deepak@BDeepaks-MacBook-Air ~ % nano task1.c
b.deepak@BDeepaks-MacBook-Air ~ %
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed.\n");
        return 1;
    }
    else if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        execlp(command, command, NULL);
        printf("Command execution failed.\n");
        exit(1);
    }
    else {
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Child process completed.\n");
    }

    return 0;
}
```

1)Then press ctrl+o enter ctrl+x2)then gcc filename.c -o filename

```
b.deepak@BDeepaks-MacBook-Air ~ % nano task1.c
b.deepak@BDeepaks-MacBook-Air ~ % gcc task1.c -o task1
b.deepak@BDeepaks-MacBook-Air ~ %
```

3)then ./filename

```
b.deepak@BDeepaks-MacBook-Air ~ % nano task1.c
b.deepak@BDeepaks-MacBook-Air ~ % gcc task1.c -o task1
b.deepak@BDeepaks-MacBook-Air ~ % ./task1
Enter a Linux command:
```

4)enter the linux command

```
b.deepak@BDeepaks-MacBook-Air ~ % ./task1
Enter a Linux command: ls
Parent PID: 40439
Child PID: 40440
a.out          Documents      first.c        Music          Pictures       task1
Applications   Downloads     Library        os             Public        task1.c
Desktop        Exp:7 DDCA.circ Movies         os.c          PyCharmMiscProject vs code
Child process completed.
b.deepak@BDeepaks-MacBook-Air ~ %
```

Child process completed.

Task 2

Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

An operating system is system software that controls all the hardware and software resources of a computer. Users do not communicate directly with the hardware. Instead, the operating system acts as a bridge between the user, applications, and the hardware

Linux Commands Used

1. uname

The `uname` command is used to display information about the Linux operating system. It provides details such as the operating system name, kernel version, machine architecture, and hostname. This command helps us identify the system on which Linux is running.

```
b.deepak@BDeepaks-MacBook-Air ~ % uname
Darwin
b.deepak@BDeepaks-MacBook-Air ~ %
```

2. lscpu

The `lscpu` command displays detailed information about the processor. It shows the CPU model, number of cores, number of threads, CPU architecture, and processor speed. This information helps us understand the hardware capabilities of the computer.

```
b.deepak@BDeepaks-MacBook-Air ~ % ./task1
Enter a Linux command: lscpu
Parent PID: 40607
Child PID: 40608
Command execution failed.
Child process completed.
b.deepak@BDeepaks-MacBook-Air ~ %
```

3. ps

The `ps` command lists the processes that are currently running in the system. It displays the Process ID (PID), Parent Process ID (PPID), user name, and the command associated with each process. This command is useful for monitoring and managing processes.

```
b.deepak@BDeepaks-MacBook-Air ~ % ps
  PID TTY          TIME CMD
 40289 ttys000    0:00.06 -zsh
b.deepak@BDeepaks-MacBook-Air ~ %
```

4. top

The top command provides a real-time view of the system's performance. It continuously displays CPU usage, memory usage, running processes, and system load. It is commonly used to identify programs that consume a large amount of system resources.

```
Processes: 865 total, 3 running, 862 sleeping, 3332 threads
Load Avg: 2.50, 2.17, 1.99 CPU usage: 1.22% user, 1.97% sys, 96.80% idle SharedLibs: 989M resident, 175M
PhysMem: 15G used (1760M wired, 1818M compressor), 587M unused. VM: 360T vsize, 6144M framework vsize, 0(0)
Disks: 8736726/249G read, 11272689/178G written.
```

PID	COMMAND	%CPU	TIME	#TH	#WQ	#PORT	MEM	PURG	CMPRS	PGRP	PPID	STATE	BOOSTS
0	kernel_task	7.0	04:23:37	629/10	0	0	25M	0B	0B	0	0	running	*0[0]
40468	top	5.4	00:01.22	1/1	0	29	10M	0B	0B	40468	40289	running	*0[1]
588	WindowServer	4.6	07:03:07	24	6	4430+	617M-	137M+	114M	588	1	sleeping	*0[1]
592	runningboard	1.9	25:38.56	7	6	1320-	12M	0B	1376K	592	1	sleeping	*4-[1]
40213	Pages	1.3	02:34.41	12	5	708	522M	21M	0B	40213	1	sleeping	*0[623+]
648	mDNSResponde	1.2	32:08.70	3	1	101	9585K	0B	736K	648	1	sleeping	*0[1]
670	com.apple.Dr	1.1	60:06.53	8	6	1402	32M	0B	2864K	670	1	sleeping	*0[1]
40070	Google Chrom	0.7	00:14.01	19	1	131	42M	0B	15M	40065	40065	sleeping	*0[3]
40196	WhatsApp	0.4	01:09.25	32	6	1075	278M	23M	3216K	40196	1	sleeping	*380[3]
40287	Terminal	0.3	00:08.12	8	3	326	107M-	42M	0B	40287	1	sleeping	*0[1023+]
40237	Safari	0.2	00:12.68	8	3	608-	48M-	70M	0B	40237	1	sleeping	*0[979+]
555	launchservic	0.2	01:32.44	5	4	1723-	8720K-	0B	2416K	555	1	sleeping	*2761+[1630]
40309	Google Chrom	0.1	00:12.58	23	1	245	227M+	16K	0B	40065	40065	sleeping	*0[15]
512	logd	0.1	11:04.60	4	3	2819	14M	0B	14M	512	1	sleeping	*0[1]
602	systemstatus	0.1	00:11.31	3	2	73+	4736K+	0B	896K	602	1	sleeping	*0[5778]
580	corebrightne	0.1	05:46.23	2	1	168	5152K	0B	1568K	580	1	sleeping	*0[1]
40470	screencaptur	0.1	00:00.08	4	2	180-	8912K-	0B	0B	40470	1	sleeping	*0[83+]
891	distnoted	0.1	01:35.41	3	2	678	5232K	0B	1648K	891	1	sleeping	*0[1]
1067	ControlCente	0.1	16:11.67	8	3	739	67M	384K	18M	1067	1	sleeping	*0[69963+]
567	distnoted	0.1	01:08.58	3	2	258+	2432K+	0B	768K	567	1	sleeping	*0[1]
963	AXVisualSupp	0.1	08:11.15	4	1	342-	18M-	0B	8352K	963	1	sleeping	*21[51]
560	locationd	0.1	12:17.22	5	2	325	11M	128K	2480K	560	1	sleeping	*0[102238]
38819	mysqld	0.1	00:24.40	38	0	61	492M	0B	488M	38716	38716	sleeping	*0[1]
982	callservices	0.1	00:44.30	5	1	398-	15M-	0B	7344K	982	1	sleeping	*0[83070+]
1215	Spotlight	0.1	03:08.11	6	3	595-	190M-	28M	94M	1215	1	sleeping	*193[29]
40065	Google Chrom	0.1	00:28.30	43	2	730	139M	0B	34M	40065	1	sleeping	*0[872+]
25359	nsattributed	0.1	00:07.18	5	1	572-	11M	0B	2992K	25359	1	sleeping	*584[4]
1	launchd	0.0	29:00.66	3	2	5113-	26M-	0B	4960K	1	0	sleeping	*0[0]
587	loginwindow	0.0	20:43.44	5	2	811-	76M	0B	19M	587	1	sleeping	*602[4902]
972	rapportd	0.0	01:22.51	3	2	255	14M	0B	6400K	972	1	sleeping	*0[1]
1207	AppSSODaemon	0.0	00:05.20	3	1	135-	5600K	0B	1840K	1207	1	sleeping	*0[8133+]
1161	CursorUIView	0.0	04:26.82	5	3	268	17M	0B	6336K	1161	1	sleeping	*0[20080+]
1109	Notification	0.0	02:57.11	5	3	432	65M	0B	35M	1109	1	sleeping	*0[15108+]
1142	SystemUIServ	0.0	00:23.84	3	1	452-	13M-	0B	3888K	1142	1	sleeping	*0[9772+]
1623	SiriNCService	0.0	01:11.04	6	2	418	26M	0B	16M	1623	1	sleeping	*0[191203+]
578	notifyd	0.0	02:05.44	2	1	1218-	4432K	0B	544K	578	1	sleeping	*0[1]
652	audioclocksy	0.0	02:28.71	3	2	53	2960K	0B	1200K	652	1	sleeping	*0[1]
947	WindowManage	0.0	08:19.46	5	2	459-	19M-	0B	3376K	947	1	sleeping	*0[166161+]
943	UserEventAge	0.0	01:25.83	3	2	1683-	9873K+	0B	3872K	943	1	sleeping	*0[1]
526	powerd	0.0	04:23.82	3	2	163	5536K	0B	656K	526	1	sleeping	*0[1]